

LEVELS 13, 21-23

Chapter 14: Interview Framework & Communication

System Design Master Roadmap — 9 Years of Experience Edition

Compiled August 2026



Chapter 14: The Interview Framework, Communication, and Seniority

← [Chapter 13: Architecture Patterns](#) · [Index](#) · Next → [Chapter 15: Diagramming Guide](#)

This is arguably the highest-leverage chapter in the entire roadmap. Everything in Chapters 1–13 is raw material; this chapter is the operating system that turns that material into a passing interview performance. Read it more than once — ideally right before every mock interview in Chapter 28.

14.1 The Universal 10-Step Framework

Use this sequence for *any* unfamiliar system design question. It's not a rigid script — a 45-minute interview won't give equal time to all ten steps, and you'll loop back between steps naturally — but it's the checklist that ensures you never skip something interviewers are silently scoring (Chapter 31's rubric maps almost exactly onto these ten steps).

Step 1 — Clarify Requirements (3–5 minutes)

What to actually say: "Before I start designing, I want to make sure I understand the scope. Let me ask a few questions." Then ask about:

- **Functional scope:** "Which specific features are we designing? For something like [Instagram], are we covering the full product, or should I focus on the feed and upload path specifically?"
- **Scale:** "Roughly how many users, and what's the read/write pattern?" (If they don't know / want you to assume, say what you're assuming out loud and move on — don't stall waiting for a number they may not have.)
- **Non-functional priorities:** "Is strong consistency required anywhere specific, or is eventual consistency acceptable for most of this? Any latency targets I should design around?"

Why this step is scored so heavily: an interviewer watching you ask sharp, scoped clarifying questions learns in 3 minutes whether you'll build the *right* system, not just a system. Skipping this is the single most commonly cited mistake in real interview feedback (Chapter 23).

What NOT to do: don't ask more than 5–6 questions — this isn't a deposition. Ask enough to scope correctly, then move.

Step 2 — Estimate Scale (3-5 minutes)

What to say: "Let me do some quick back-of-envelope numbers to ground the design." Then walk through DAU → RPS → peak RPS → storage → bandwidth, out loud, rounding aggressively (Chapter 3's full framework). Narrate your rounding: "I'll round 86,400 seconds in a day to about 100,000 for quick mental math."

Why: this is where you *earn* every architecture decision that follows — "I need a cache" means something concrete once you've said "12,000 reads/sec, mostly hitting the same popular content."

Step 3 — Define the API (3-5 minutes)

What to say: "Let me sketch the core API surface before diving into architecture." List 4-6 key endpoints with method, path, and key params — don't over-specify every field. `POST /orders`, `GET /orders/{id}`, `GET /users/{id}/feed?cursor=...`. Mention idempotency keys where relevant (Chapter 6.4/9.4) — this is a small detail that reliably impresses.

Step 4 — Define the Data Model (5 minutes)

What to say: "Here are the core entities and their relationships." Sketch the main tables/collections with key fields and relationships — not a full schema, just enough to show you understand the data shape and have made a real SQL-vs-NoSQL decision (Chapter 4.6) with a stated reason.

Step 5 — High-Level Architecture (5-8 minutes)

What to say: "Let me put this together into an architecture." Draw boxes and arrows (Chapter 15's notation): client → CDN/LB → API Gateway → services → cache/DB → async workers/queue. Narrate the request flow as you draw: "A user request comes in, hits the load balancer, gets routed to the order service, which checks the cache first, falls back to the database on a miss..."

Step 6 — Deep Dive (10-15 minutes — usually the bulk of the interview)

The interviewer will typically steer you toward 1-2 specific areas they care about most (this is often a deliberate test of whether you follow their lead or bulldoze your own plan). Be ready to go deep on: database choice and scaling, caching strategy, queue/async processing, consistency model for this specific data, and failure handling. **Say explicitly:** "Which area would you like me to go deeper on?" — inviting the interviewer to steer is itself a strong senior signal (it shows collaborative interviewing instead of a rehearsed monologue).

Step 7 — Identify Bottlenecks (3-5 minutes)

What to say: "If I think about where this design would break first under 10x load, I'd point to..." — then name a specific bottleneck (a single database write path, a hot partition, a synchronous call chain) and how you'd address it. This step is frequently skipped by candidates who run out of time because they over-explained Step 5 — budget for it explicitly.

Step 8 — Reliability (3-5 minutes)

What to say: "For failure handling, here's what I'd add..." — timeouts, retries with backoff/jitter, circuit breakers, replication, and (if relevant to scope) a disaster recovery posture (RPO/RTO from Chapter 11).

Step 9 — Trade-offs (woven throughout, but worth an explicit recap)

What to say, near the end: "To recap the key trade-offs I made: I chose eventual consistency for the feed to prioritize latency and availability, but strong consistency for the payment ledger because correctness matters more there than speed." Naming your trade-offs explicitly, unprompted, is one of the highest-signal things you can do — it proves you know there *were* alternatives, not just that you know one answer.

Step 10 — Future Scaling (1-2 minutes, if time allows)

What to say: "If this needed to scale another 10x, or expand to multi-region, here's what I'd revisit first..." — a brief, confident closing note, not a new deep dive.

14.2 Interview Communication — Bad, Better, Excellent

Situation: The interviewer asks "Design Instagram" with no other context

Bad: *[Immediately starts drawing]* "Okay so we'll need a load balancer, then some app servers, then a database, and we'll use Redis for caching, and Kafka for..."

Why it's bad: zero clarification, jumping straight to technology names before understanding the problem — the interviewer has no idea if you understood what "Instagram" even means to them in this context (photo feed? DMs? Reels? all of it?).

Better: "Sure — so we need users to be able to upload photos, follow each other, and see a feed. I'll assume we're at a scale of a few hundred million daily users. Let me start with the API design..."

Why it's better: states assumptions, but doesn't actually check them with the interviewer — a monologue, not a conversation.

Excellent: "Happy to design Instagram — that's a big surface area, so let me scope it with you first. Are we covering the full product, or should I focus on a specific slice — say, the upload-and-feed path, which is usually the meatiest part? [interviewer responds] Great. And should I assume this is at Instagram's actual current scale — hundreds of millions of DAU — or a smaller target you have in mind? [interviewer responds] Okay, I'll estimate scale quickly, then walk through the API, data model, and architecture, and we can go deep wherever you'd like."

Why it's excellent: treats it as a conversation, scopes explicitly, sets expectations for how the next 40 minutes will flow — which also subtly puts you in control of pacing.

Situation: You realize partway through that your database choice is wrong for a requirement you missed

Bad: *[Silently panics, tries to awkwardly retrofit the wrong DB without acknowledging it]*

Better: "Actually, wait, let me reconsider this — I don't think MongoDB is right here."
[Switches without explaining why]

Excellent: "Actually, now that we've discussed the need for multi-record transactions across orders and inventory, I want to revise my earlier database choice — I initially reached for MongoDB, but given this specific requirement, a relational database with real ACID transactions is a better fit. This is exactly the kind of thing I'd want to catch before writing code, so I'm glad we talked through the requirement in enough detail to surface it now."

Why it's excellent: revising a decision openly, with a clear reason, is a **positive** signal, not a negative one — it shows you update your design based on new information instead of defending an initial guess out of ego. Interviewers explicitly look for this.

Situation: The interviewer challenges your design — "why not just use a single big database instead of all this complexity?"

Bad: "No, we need microservices and sharding because that's how it's done at scale."
(defensive, no reasoning)

Better: "Because a single database won't scale to the numbers we calculated earlier."
(true, but thin — doesn't re-engage with their specific challenge)

Excellent: "That's a fair challenge, and honestly, for a lot of systems a single well-tuned database with read replicas would be the *right* answer, not over-engineering — I want to be clear I'm not defaulting to complexity for its own sake. In this specific case though, we

calculated write throughput around [X] and data volume around [Y], which is past what a single primary comfortably sustains long-term, and that's the specific signal that justifies the extra complexity here. If those numbers were 10x smaller, I'd genuinely recommend the simpler single-database design instead."

Why it's excellent: engages directly with the challenge, references concrete numbers instead of general principle, and — crucially — concedes the interviewer's underlying point (simpler is often better) while explaining precisely why *this* case is the exception. This response pattern (validate the general principle, then justify the specific exception with evidence) is reusable across almost any "why didn't you do the simpler thing" challenge.

Situation: You genuinely don't know something

Bad: *[Makes something up confidently] or [goes silent for 30 seconds]*

Better: "I'm not 100% sure about the exact internals there, let's move on."

Excellent: "I don't know the exact internal mechanism Kafka uses for that specific edge case off the top of my head — but here's how I'd reason about it and what I'd verify: [reasons through it from first principles, states the assumption clearly, and notes what they'd check/test in a real system]. I'd rather be explicit that this is a reasoned guess than state it as fact."

Why it's excellent: honesty about the boundary of your knowledge, paired with a demonstration that you can still reason productively past that boundary, is a stronger signal than false confidence — and vastly stronger than genuinely bluffing, which experienced interviewers catch more often than candidates expect.

Situation: Avoiding overengineering

Bad (overengineered): For a simple internal admin tool: "I'd use Kafka for event streaming, deploy on Kubernetes with a service mesh, use CQRS with event sourcing for the audit trail, and shard the database across three regions."

Excellent: "Given this is a low-traffic internal tool used by maybe 50 employees, I'd keep this deliberately simple — a modular monolith, a single well-indexed PostgreSQL database, deployed as one service behind a load balancer for redundancy. None of the scale-driven complexity — sharding, event streaming, microservices — is earned by the actual requirements here, and adding it would just be slower to build and harder to operate for no real benefit. I'd revisit this if usage patterns changed dramatically, but I wouldn't design for a hypothetical scale that isn't in scope."

Why it matters: knowing when *not* to use advanced patterns is scored just as highly as knowing when to use them — see Chapter 23 for the full "jumping to Kafka immediately" mistake pattern.

14.3 Junior vs. Mid vs. Senior vs. Staff — What Actually Changes

Given your 9 years of experience, you should be operating at the senior tier and reaching for staff-level moments — here's precisely what distinguishes each level in how they'd answer the same question, "design a URL shortener's storage layer":

Level	Typical answer shape
Junior	"I'd use a database table with columns for the short code and long URL." Correct but shallow — no scale reasoning, no discussion of how the short code is generated, no mention of read/write patterns.
Mid-level	"I'd use a relational database, generate the short code via a hash or base62 encoding of an auto-incrementing ID, and add a cache in front since reads outnumber writes." Correct and reasonably complete — knows the standard pattern, applies it.
Senior	Everything mid-level does, <i>plus</i> explicit trade-off articulation: "I'd choose base62 encoding of an auto-incrementing ID over a random hash, because it guarantees no collisions without needing a uniqueness check, at the cost of short codes being sequentially guessable — which I'd mitigate if that's a real concern for this product. I'd shard the ID generator itself if a single sequence became a bottleneck, using something like Twitter's Snowflake approach — reserved ID ranges per node." Reasons about trade-offs and names the next scaling step <i>before being asked</i> .
Staff	Everything senior does, <i>plus</i> business and organizational context: "I'd also flag that the choice between sequential and random short codes has a product-security dimension — sequential codes leak approximate creation order and volume to anyone paying attention, which might matter if this is used for anything semi-sensitive, so I'd loop in product/security on that trade-off rather than deciding it unilaterally as a backend concern. I'd also design the ID-generation service as an internally reusable component from day one, since 'generate a unique, distributed ID' is a need that resurfaces in almost every other service this org will build." Connects the technical decision to business/security/organizational impact, and thinks about reusability and cost beyond the immediate problem.

The practical takeaway for your prep: in every mock interview (Chapter 28), after you answer, explicitly ask yourself "did I just give a mid-level or a senior-level version of that answer?" — the difference is almost always **unprompted trade-off articulation** and **proactively naming the next scaling step**, not additional technical facts.

14.4 Common Mistakes and Their Fixes

Mistake	Why it hurts you	The fix
Jumping straight to Kafka/microservices/sharding	Signals pattern-matching on buzzwords rather than deriving the need from requirements	Always state the <i>evidence</i> (a number from your capacity estimate, a specific

Mistake	Why it hurts you	The fix
		requirement) that justifies each piece of complexity
Using microservices for everything, even small systems	Shows you don't understand the real cost of the pattern you're reaching for	Default to a modular monolith unless the scale/team-structure numbers justify otherwise (Chapter 2.8)
"I'd use Redis" with no justification	A name-drop, not a design decision	Always pair a technology choice with the specific problem it solves <i>for this system</i> — "Redis, because reads outnumber writes 100:1 and the hot set fits comfortably in memory"
No capacity estimation	Every subsequent decision is unanchored, and the interviewer can't verify your reasoning	Always do Step 2 of the framework, even briefly, even if rough
No requirements clarification	You might design the wrong system entirely, however well you design it	Always do Step 1, budget 3–5 minutes, don't skip it under time pressure
No failure handling discussed	A design that only works in the happy path isn't a senior-level design	Reserve explicit time for Step 8, even if brief
Ignoring consistency requirements	A common, serious correctness gap, especially painful in fintech-adjacent designs	Explicitly state the consistency model per data type, not once for the whole system
Ignoring security entirely	A real deduction at fintech and Tier-1 companies specifically	Weave in 1–2 concrete security decisions naturally, don't need a dedicated monologue
Ignoring observability	Signals you've never operated a system in production, which is at odds with your actual experience	Mention logging/metrics/tracing briefly, especially given this is genuinely your strength — use it
Overengineering a simple system	Just as bad a signal as underengineering a complex one	Explicitly size the solution to the stated scale — say "this doesn't need X because Y" out loud
No trade-offs stated	The single highest-value fix in this whole table — almost every design decision has one worth naming	End major decisions with "the trade-off here is..." as a habit, not an afterthought
Unclear/messy diagrams	Interviewers use your diagram as a proxy for how clearly you think	Follow Chapter 15's notation and build it incrementally, narrating as you go
Not explaining bottlenecks	Skips Step 7 entirely, missing an easy scoring opportunity	Reserve explicit time near the end even if you're running long elsewhere

Chapter 14 Interview Drill

1. Recite the 10-step framework from memory, in order, with one sentence of what you'd say at each step.
2. Practice the "excellent" response to a challenge on your design out loud, using your own words, for a design you've studied.
3. Take one of your own past answers (real or mock) and identify whether it was mid-level or senior-level, and what one sentence would upgrade it.
4. Pick three mistakes from the table above that you personally are most likely to make, based on your background, and write a one-line reminder for each.

Next → [Chapter 15: Diagramming Guide](#) — *standard notation, request/data/async/failure flows, and how to draw a clean interview diagram under time pressure.*